

Microservices Data Management: Commands and Queries

Introduzione

Dove siamo

Lezione	Focus
10	Scegliere il database giusto: Database-per-Service, polyglot persistence, CAP, sharding
11 ← siamo qui	Comandi e query : come i microservizi interagiscono con dati distribuiti
12	Transazioni distribuite
★ Le soluzioni già in mano — e perché non bastano	

Prima di introdurre i nuovi pattern, la lezione fa un'analisi onesta di **cosa abbiamo già imparato e perché non risolve il problema**:

Opzione già nota	Perché non basta
1. Comunicazione sincrona HTTP diretta	<i>"Non è una buona soluzione: crea accoppiamento tra microservizi e fa perdere il potere dell'indipendenza"</i>
2. Comunicazione asincrona (Lez. 9)	★ <i>"Ma queste query richiedono una risposta immediata, quindi non possiamo procedere con la comunicazione asincrona. L'async disaccoppia le chiamate, ma il servizio chiamante deve comunque attendere sincronamente la risposta"</i>
3. Service Aggregator (Lez. 8)	<i>"Aggrega le query nell'API Gateway, ma comporta un aumento dell'accoppiamento e della latenza"</i>

★ **Il punto chiave**: la comunicazione asincrona funziona benissimo per i **comandi** (l'utente non aspetta), ma **non per le query** — dove "il client invia richieste di query ai microservizi interni per accumulare dati nella pagina front-end, e queste richieste hanno bisogno di una risposta immediata".

"La soluzione che elimina questo tipo di dipendenze cross-service è il pattern delle viste materializzate."

Gli esempi che illustrano il problema

Caso semplice — "Ottieni l'articolo con i dettagli":

Basket —sync—→ Catalog (nome prodotto)

—sync—→ Pricing (prezzo)

"Solo due chiamate interne. Può essere gestibile e accettabile per alcuni sistemi."

Caso complesso — "Ottieni l'ordine con tutti i dettagli":

"Ci sono sei richieste HTTP sincrone concatenate per una singola richiesta del client. È evidente che questo aumenta la latenza e influisce negativamente su prestazioni, scalabilità e disponibilità."

"Cosa succede se il passo 5 o 6 fallisce? O se alcuni servizi sono inattivi — o anche solo occupati e incapaci di rispondere tempestivamente, causando latenze non accettabili?"

Materialized View Pattern

Definizione

Un microservizio memorizza la propria copia locale, denormalizzata e di sola lettura, dei dati che gli servono frequentemente da altri servizi. Queste copie locali precalcolate si chiamano viste materializzate (o read model).

Come funziona nel nostro caso

Prima (chiamate cross-service):

```
Basket —HTTP—> Catalog (per ogni item: nome)
      —HTTP—> Pricing (per ogni item: prezzo)
```

Dopo (vista materializzata):

```
Basket DB
├─ carts / cart_items
└─ product_view (VISTA MATERIALIZZATA) ← copia locale denormalizzata
    productId | productName | currentPrice
```

*"Invece di dover interrogare Catalog e Pricing ogni volta, **Basket mantiene la propria tabella all'interno del proprio database**. Questa tabella conterrebbe una copia denormalizzata dei dati essenziali di cui ha bisogno: productId, productName, currentPrice."*

*"Quando Basket ha bisogno di visualizzare queste informazioni, **interroga semplicemente la propria vista materializzata locale, che è estremamente veloce.**"*

★ *"Copiando i dati necessari direttamente nel data store di Basket, eliminiamo la necessità di quelle costose chiamate sincrone cross-service per le operazioni di lettura. L'intera operazione può essere eseguita interrogando un'unica fonte di dati locale."*

I 4 vantaggi

Vantaggio	Dettaglio
Prestazioni e latenza	"I dati sono locali e spesso denormalizzati o pre-aggregati in un formato ideale. Nessun hop di rete verso altri servizi "
Ridotto accoppiamento	"Basket non ha più una dipendenza a runtime da Catalog o Pricing per le letture"
★ Dati ottimizzati per le query	"Possono contenere dati pre-joinati, campi calcolati, o dati aggregati da più fonti "
Ridotto carico sui servizi	"Catalog e Pricing riceveranno molte meno richieste dirette,

Vantaggio	Dettaglio
di origine	liberando le proprie risorse"

★ Le 3 considerazioni critiche

1. Duplicazione dei dati — intenzionale

*"Creando una vista materializzata, **stiamo intenzionalmente duplicando i dati. In questo caso non è un anti-pattern, perché è una scelta strategica per ottimizzare le letture e migliorare la resilienza.**"*

Questo è importante: la duplicazione dei dati è normalmente da evitare (DRY), ma qui è **deliberata e giustificata**.

2. Sincronizzazione e staleness

"La domanda più critica: come e quando vengono aggiornate le viste materializzate quando cambia l'origine dei dati? Se il prezzo cambia in Catalog, la copia in Basket dovrà essere aggiornata. Se non lo è tempestivamente, Basket potrebbe visualizzare prezzi obsoleti."

3. Proprietà dei dati ★

"È fondamentale ricordare che la vista materializzata è di SOLA LETTURA e "usa e getta" (disposable). Il servizio originale — Catalog, Pricing — rimane l'unica fonte di verità e il proprietario di quei dati. La vista materializzata può sempre essere ricostruita dall'origine."

Questa è la regola che impedisce al pattern di degenerare: **la copia non è mai autorevole**. Se si corrompe, la butti via e la ricostruisci dagli eventi.

Come si mantiene sincronizzata

"L'approccio più comune e consigliato è quello degli aggiornamenti asincroni guidati dagli eventi."

1. Cambia il prezzo in Catalog (fonte di verità)
2. Catalog pubblica ProductPriceChangedIntegrationEvent sull'event bus
3. Basket è sottoscritto a quell'evento
4. Basket riceve l'evento e aggiorna la propria vista materializzata locale

*"Questo meccanismo asincrono significa che la vista materializzata raggiungerà l'eventual consistency: diventerà coerente **poco dopo** che la sorgente è cambiata, ma potrebbe esserci un breve ritardo. Per molti scenari di lettura, questo leggero ritardo è perfettamente accettabile in cambio dei vantaggi in prestazioni e resilienza."*

Altri metodi meno comuni: "batch job programmati o trigger di database, ma **gli aggiornamenti guidati dagli eventi tramite publish/subscribe sono generalmente preferiti** per il loro disaccoppiamento e la natura quasi in tempo reale."

Nota che questo è esattamente ciò che abbiamo già implementato nella Lezione 9: quando Basket consuma ProductPriceChangedIntegrationEvent e aggiorna

i prezzi nella cache Redis, **sta mantenendo una vista materializzata**. Il codice c'era già — ora abbiamo il nome del pattern.

CQRS: Command Query Responsibility Segregation

Definizione

CQRS separa la parte dell'applicazione che **MODIFICA** lo stato da quella che **LEGGE** lo stato.

Le due categorie:

	Command (comando)	Query (interrogazione)
Cosa fa	Modifica i dati o lo stato del sistema	Legge i dati e li restituisce senza modificare lo stato
Esempi	CreateOrder, UpdateProductPrice, AddItemToBasket	GetProductDetails, ListCustomerOrders
Restituisce dati?	"Di solito no — esegue un'azione "	Sì

*"Questi due tipi di operazioni sono gestiti da **modelli diversi, percorsi logici diversi, e potenzialmente anche data store fisici diversi**. Questa separazione ci permette di **ottimizzare ogni percorso in modo indipendente** per i suoi requisiti specifici di prestazioni, scalabilità e coerenza."*

★ Perché separare: l'asimmetria tra letture e scritture

Questa è la motivazione centrale del pattern:

*"In molte applicazioni, **le caratteristiche e i requisiti delle operazioni di lettura e scrittura sono fundamentalmente diversi**. Sono **ASIMMETRICI**."*

	Scritture (Command)	Letture (Query)
Complessità	"Complesse validazioni e logica di business "	Semplici
Coerenza	"Richiedono forte coerenza ACID "	" Eventual consistency spesso accettabile"
Modello dati ideale	" Normalizzato , per garantire integrità ed evitare ridondanza"	" Denormalizzato , o viste precalcolate ottimizzate per le query specifiche"
Frequenza	"Potrebbero essere meno frequenti , ma più critiche in termini di accuratezza "	"Devono essere molto veloci e gestire volumi elevati "
Problema tipico	—	"Devono unire dati di più tabelle , cosa lenta su database altamente normalizzati progettati per le scritture "

★ *"Cercare di soddisfare queste esigenze molto diverse con un unico database e un unico modello dati può portare a colli di bottiglia e compromessi di progettazione."*

L'esempio concreto del conflitto:

*"La necessità di **unire dieci tabelle per una vista complessa** può bloccare il database e compromettere le operazioni di scrittura. Al contrario, **validazioni complesse e logica di business sul lato scrittura** possono rallentare il database per tutti."*

I 4 vantaggi

Vantaggio	Dettaglio
Scalabilità indipendente ★	"Se l'applicazione è read-intensive , con molte più query che comandi, possiamo scalare i database di lettura in modo molto più aggressivo rispetto a quelli di scrittura"
Ottimizzazione dei modelli dati	"Un modello altamente normalizzato per l'integrità sul lato scrittura, e un modello completamente diverso — pre-aggregato e denormalizzato — sul lato lettura"
Prestazioni delle query	"Le query contro uno store di lettura con viste materializzate o dati denormalizzati possono essere estremamente veloci , evitando i join complessi"
Sicurezza e logica più semplice	"Si può applicare sicurezza più stretta al lato comandi , e la logica di query è più semplice perché lavora con read model costruiti appositamente"

I 4 svantaggi

1. Maggiore complessità

*"CQRS è un **pattern architetturale più complesso di un semplice approccio CRUD** con un unico database. Si hanno **modelli separati, fonti dati potenzialmente separate, e un meccanismo di sincronizzazione**. È **generalmente sconsigliato per applicazioni semplici**."*

2. Eventual consistency ★

*"Poiché i read model sono aggiornati in modo asincrono, ci sarà un periodo in cui i dati letti potrebbero essere **leggermente stale**. L'applicazione e gli utenti devono **tollerarlo**. Se è richiesta **forte coerenza immediata sulle letture**, CQRS con store separati potrebbe non essere adatto, o richiedere progettazione accurata."*

3. Duplicazione del codice

*"Le strutture dati potrebbero dover essere **rappresentate diversamente nel modello di scrittura rispetto ai read model**, con conseguente duplicazione o logica di trasformazione."*

4. Logica di sincronizzazione

*"Bisogna **implementare e mantenere il meccanismo che propaga le modifiche** dallo store di scrittura a quello di lettura."*

Le 3 best practice

1. Separare fisicamente i database di lettura e scrittura

*"Si può implementare CQRS con **un unico database usando viste o tabelle diverse**, ma per ottenere notevoli vantaggi di scalabilità spesso è meglio **separarli fisicamente**."*

Nota importante: CQRS ha **due livelli di intensità**. Il livello "leggero" separa solo i *modelli* (classi Command/Query) nella stessa applicazione e database — ed è utile e poco costoso. Il livello "pesante" separa i *database*, e da lì nasce tutta la complessità di sincronizzazione.

2. **Progettare lo schema di lettura ottimizzato per le query specifiche** — *"il pattern delle viste materializzate è un modo eccellente per implementare questi read model"*
3. **Usare tecnologie di database diverse** ★

*"Non abbiate paura di usare tecnologie diverse per lettura e scrittura. Ad esempio: **PostgreSQL per il lato scrittura**, per garantire conformità ACID e integrità; **un document DB NoSQL o un key-value store per i read model**, per ottimizzare la velocità delle ricerche."*

L'esempio Instagram (anticipazione)

*"Instagram usa **PostgreSQL** per dati utente principali e metadati, **che richiedono forte coerenza** — questo è il loro **lato scrittura**. Ma per **feed utente, flussi di attività e visualizzazioni di storie** — **estremamente read-heavy**, che devono scalare massicciamente, e **dove l'eventual consistency è accettabile** — **sfruttano Cassandra**."*

*"La creazione di un nuovo post viene elaborata e poi **denormalizzata in questi store NoSQL ottimizzati per la lettura**. Questo approccio ibrido consente il meglio di entrambi i mondi."*

How to Sync Read and Write Database in CQRS?

Il problema

*"Quando implementiamo CQRS con **data store fisicamente separati** — uno ottimizzato per le scritture (**la nostra fonte di verità**) e uno o più ottimizzati per le letture (**i read model / viste materializzate**) — **la sfida centrale diventa la sincronizzazione**."*

"Se il prezzo cambia nel database di scrittura, come fa il database di lettura a ricevere l'informazione aggiornata, così che le query successive riflettano i nuovi prezzi?"

⚠ Cosa NON fare:

*"Se si cerca di **aggiornare entrambi sincronamente nella stessa transazione**, si **vanifica lo scopo di CQRS**, reintroducendo accoppiamento e potenziali problemi di prestazioni."*

★ La soluzione: sincronizzazione event-driven

*"La soluzione più comune e generalmente consigliata è usare un'architettura **event-driven con messaggistica asincrona**."*

1. Un COMANDO modifica con successo i dati nel DB DI SCRITTURA (fonte di verità)
 - ↓
2. La logica applicativa PUBBLICA UN EVENTO che descrive il cambiamento (es. ProductPriceUpdatedEvent, contenente i dati rilevanti)
 - ↓
3. L'evento va sul MESSAGE BROKER (RabbitMQ / Kafka)
 - ↓
4. Gli EVENT CONSUMER dedicati – processi separati o parte della logica che gestisce gli store di lettura – sono sottoscritti
 - ↓
5. Il consumer AGGIORNA i database di lettura / viste materializzate

"L'intero processo di sincronizzazione è asincrono. Il comando originale viene completato dopo l'aggiornamento del DB di scrittura e la pubblicazione dell'evento. NON aspetta che tutti i read model siano aggiornati."

L'implicazione: eventual consistency

*"Poiché gli aggiornamenti dei DB di lettura avvengono **dopo** l'aggiornamento del DB di scrittura e **in risposta a un evento**, ci sarà inevitabilmente un breve periodo — millisecondi, o alcuni secondi in sistemi ad alto carico — durante il quale il DB di lettura potrebbe non riflettere ancora le ultime modifiche."*

"Durante quella breve finestra, le query che colpiscono il DB di lettura potrebbero restituire il vecchio prezzo. Ecco cosa intendiamo per eventual consistency: il DB di lettura rifletterà le modifiche, ma non necessariamente immediatamente."

*"È un trade-off molto comune nei sistemi distribuiti, e una conseguenza spesso discussa del **teorema CAP**. Per ottenere disaccoppiamento, prestazioni e scalabilità, l'applicazione e potenzialmente l'interfaccia utente devono essere progettate per tollerare questo livello di coerenza."*

Le 3 tecniche alternative di sincronizzazione

Tecnica	Come funziona	Quando
Database Replication	"Si usano le funzioni di replica integrate del database per creare repliche di sola lettura . Le query vengono indirizzate a queste repliche"	"Per scenari più semplici, dove il read model può condividere la stessa tecnologia e uno schema simile . Spesso usato per scalare le letture dei database relazionali "
CDC — Change Data Capture ★	"Invece di pubblicare esplicitamente eventi dall'applicazione, gli strumenti CDC monitorano il transaction log del database di scrittura e pubblicano automaticamente eventi per qualsiasi modifica "	"Permette di disaccoppiare ulteriormente la logica applicativa dalla pubblicazione degli eventi "
Event Sourcing	"Non si memorizza lo stato corrente, ma una sequenza di tutti gli eventi accaduti a un'entità . Lo stato attuale si ricava riproducendo gli eventi , e questi stessi eventi sono perfetti per costruire e aggiornare i	Vedi sezione successiva

Tecnica

Come funziona

Quando

read model"

CDC merita attenzione: strumenti come **Debezium** leggono il write-ahead log del database (il log delle transazioni) e ne derivano eventi. Il vantaggio è che **non puoi dimenticarti di pubblicare un evento** — qualsiasi modifica al database genera un evento per costruzione. Torna nella Lezione 12.

Event Sourcing Pattern with CQRS and Eventual Consistency

Il cambio di paradigma

*"Nella maggior parte delle applicazioni tradizionali, **quando si aggiornano i dati si sovrascrive lo stato esistente**. Se un utente cambia email, **il vecchio indirizzo viene sostituito con quello nuovo**. Conosciamo solo lo stato attuale."*

Definizione: Event Sourcing — *"Invece di memorizzare solo lo stato corrente di un'entità, **si memorizza la sequenza completa di tutti gli eventi di cambiamento di stato che si sono verificati per quell'entità**."*

Il confronto:

APPROCCIO TRADIZIONALE (state-oriented)

users

└─ userId: 1, email: "c@x.com"

← solo lo stato ATTUALE
la storia è persa

EVENT SOURCING (event-oriented)

event_store

└─ UserCreated (email: "a@x.com") ← t0

└─ EmailChanged (email: "b@x.com") ← t1

└─ EmailChanged (email: "c@x.com") ← t2

↓ replay

stato attuale: email = "c@x.com"

Le proprietà fondamentali

- "Ogni azione o cambiamento è catturato come **un evento IMMUTABILE** che descrive **qualcosa che è accaduto**" — es. OrderCreated, ProductPriceChanged, ItemAddedToBasket
- "Gli eventi sono **accodati in ordine cronologico** in un archivio specializzato: **l'Event Store**"
- ★ "Nell'event store non viene **MAI cancellato o sovrascritto nulla**. Si aggiungono solo **nuovi eventi** (append-only)"
- "Questo log eventi diventa la **fonte di verità definitiva** per la storia di quell'entità"

Come si ottiene lo stato corrente

"Per ottenere lo stato attuale di un'entità, si riproducono (replay) tutti gli eventi associati dall'event store in ordine cronologico. Ogni evento modifica lo stato, portando allo stato attuale."

I 3 superpoteri dell'Event Sourcing

1. Audit trail completo e immutabile ★

*"L'event store fornisce naturalmente **un registro di controllo completo e immutabile di OGNI modifica dei dati**. Prezioso per debug, verifica e comprensione del comportamento del sistema nel tempo."*

2. Query temporali / time travel ★

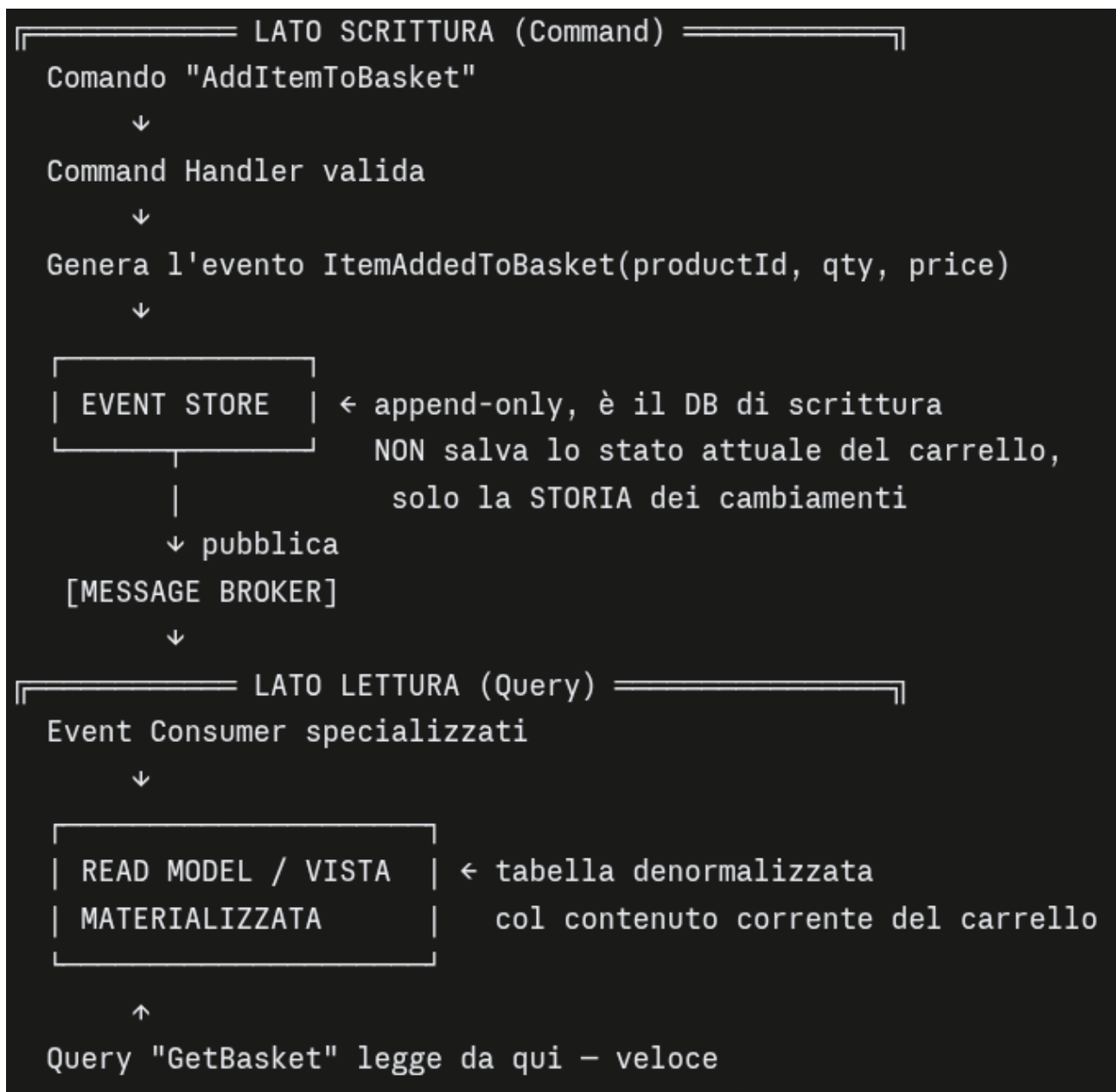
*"Poiché si dispone della storia completa, è **possibile ricostruire lo stato di un'entità come era in QUALSIASI momento del passato**, riproducendo gli eventi fino a quel momento specifico. Questo consente potenti analisi storiche."*

3. Alimentazione naturale dei read model

*"Gli eventi persistiti **non servono solo a ricostruire lo stato**: possono anche essere pubblicati su un event bus. Altri servizi possono sottoscriverli per **aggiornare i read model denormalizzati (viste materializzate)** o attivare altri flussi di lavoro."*

★ CQRS + Event Sourcing: l'accoppiata naturale

"È un'accoppiata molto comune e naturale nelle architetture event-driven."



Lato comandi:

"Quando arriva un comando, invece di aggiornare direttamente una tabella che rappresenta lo stato attuale, la logica GENERA uno o più eventi. Questi eventi vengono accodati all'event store, che diventa il nostro database di scrittura. Memorizza solo eventi immutabili — non viene salvato lo stato attuale del carrello, ma solo la storia delle modifiche."

Lato query:

"Le viste materializzate che servono le query sono costruite consumando gli eventi pubblicati dall'event store. Quando i consumer ricevono un evento come ItemAddedToBasket, aggiornano il loro read model specifico — ad esempio una tabella denormalizzata che mostra il contenuto corrente del carrello per una rapida visualizzazione."

★ *"Questa architettura consente al lato scrittura di essere ottimizzato per l'append rapido di eventi, e al lato lettura di essere ottimizzato per query veloci, con scalabilità indipendente per entrambi."*

Il trade-off finale

"Le scritture sono veloci e fortemente coerenti all'interno dell'event store per quello specifico stream. Ma poiché l'aggiornamento del lato lettura è asincrono, ci sarà un ritardo. Per un breve periodo, una query su un read model potrebbe restituire dati che non riflettono ancora l'ultimo evento appena persistito."

"Questo è il trade-off fondamentale: otteniamo enorme scalabilità, flessibilità nei read model, un audit trail completo e scritture disaccoppiate — ma accettiamo che le nostre viste di lettura possano essere leggermente stale per un tempo molto breve."

Design Instagram System & Database Architecture

Un case study reale che mette insieme **tutto** quello visto nelle Lezioni 10 e 11.

I numeri

"Centinaia di milioni di persone lo visitano quotidianamente. Circa 100 milioni di foto e 400 milioni di storie condivise ogni giorno. Miliardi di like generati."

L'architettura generale

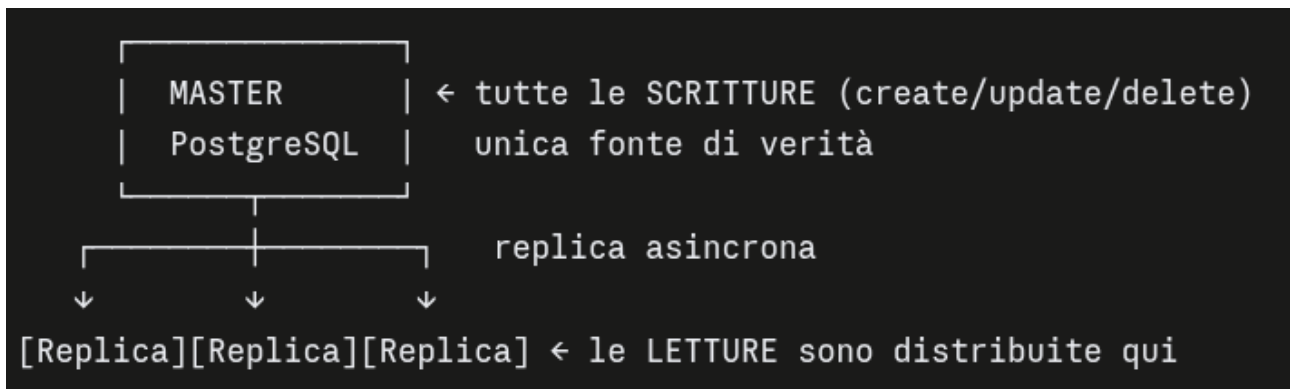
- **Più data center** strategicamente posizionati → *"gli utenti si collegano al data center geograficamente più vicino, riducendo la latenza"*
- **Load balancer** che distribuiscono su una flotta di **web/application server** — *"Instagram usa notoriamente Django (Python)"*
- **Caching esteso** — **Memcached**
- **Code asincrone** — *"qualcosa come RabbitMQ o Celery per l'elaborazione in background"*

★ La polyglot persistence

"Non si affidano a un'unica tecnologia. Hanno due database principali: PostgreSQL e Apache Cassandra."

PostgreSQL — il lato SCRITTURA

Configurazione: replica master-slave



Cosa memorizza:

- Informazioni principali degli utenti (username, dettagli profilo, **password hashate**)
- **Metadati dei media** (didascalie, tag, posizioni)
- ⚠ **Non** i file veri: "immagini e video sono memorizzati in **un object store separato**, e PostgreSQL memorizza **i puntatori/link** ad essi"

"Questo consente a Instagram di **mantenere forte coerenza per le scritture critiche**, scalando al contempo le letture tramite le repliche."

Cassandra — il lato LETTURA ad alta scala

Architettura: "peer-to-peer masterless. A differenza del master-slave di PostgreSQL, ogni nodo è uguale all'altro. Ogni nodo può gestire letture o scritture per i dati di cui è responsabile. Questo elimina il single point of failure e semplifica lo scaling."

Sharding automatico: "Quando si definisce il modello dati si sceglie una **partition key**, che Cassandra usa per distribuire i dati uniformemente su tutti i nodi."

Cosa memorizza:

- **Feed utente** — "per generare un home feed personalizzato per centinaia di milioni di utenti, mostrando i post recenti delle persone che seguono: **un'operazione incredibilmente pesante in lettura e scrittura**"
- **Activity feed e storie**
- **Messaggistica diretta**

"Cassandra è stato scelto per **scalabilità orizzontale estrema, alta disponibilità (dati replicati su più nodi), eccellenti prestazioni di scrittura**. E ha ottenuto questo abbracciando l'**eventual consistency**, allineandosi al sistema AP del teorema CAP."

★ I 3 principi architetturali di Instagram

1. Polyglot Persistence — PostgreSQL per forza relazionale e ACID; Cassandra per scalabilità e prestazioni di scrittura

2. CQRS

"PostgreSQL gestisce le **scritture critiche** che necessitano forte coerenza; Cassandra gestisce volumi di lettura estremamente elevati dove l'**eventual consistency** è accettabile. Questa separazione dei percorsi ottimizzati per lettura e scrittura è il fulcro di CQRS."

La decisione consapevole:

*"La scelta di usare Cassandra per i feed è **una decisione architetturale deliberata**: in un feed utente è **più importante che sia sempre disponibile e veloce, anche se mostra dati obsoleti di qualche secondo.**"*

3. Scaling multidimensionale ★

*"Usano tutti gli assi dello **Scale Cube**: **asse X** per eseguire istanze multiple dei server applicativi, **asse Y** per la decomposizione funzionale, **asse Z** per lo sharding integrato in Cassandra."*

"Questo dimostra come questi pattern architetturali non siano solo teorici, ma vengano attivamente usati per realizzare alcune delle applicazioni più grandi e di successo al mondo."

Design: Microservices Architecture with CQRS, Event Sourcing, Eventual Consistency

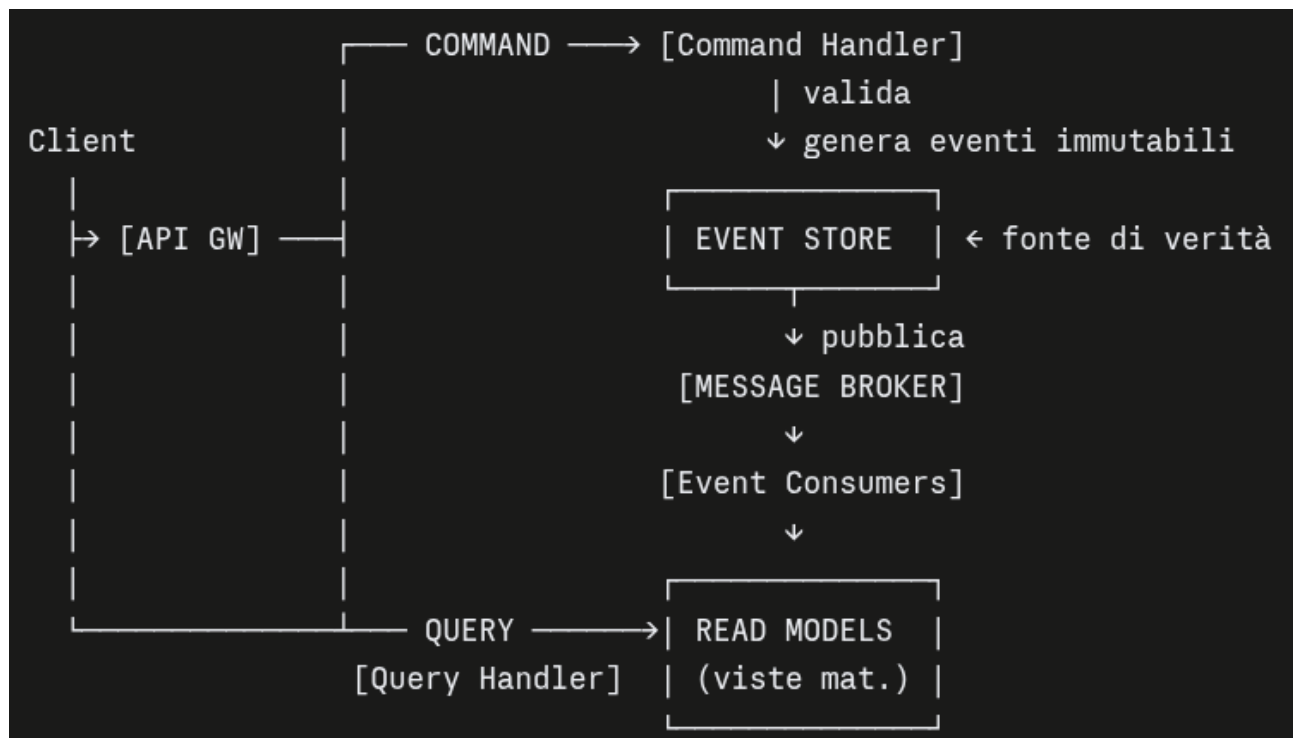
Toolkit aggiornato

Nuovi requisiti non funzionali:

1. **Alto throughput di scrittura e audit trail completo** per tutti i comandi relativi agli ordini
2. **Ottimizzazione delle prestazioni di lettura** per le varie query e viste degli ordini
3. **Scalare i percorsi di lettura e scrittura in modo indipendente**
4. **Resilienza** nella propagazione delle modifiche
5. ★ **Eventual consistency accettabile** per i read model derivati dal lato scrittura

Nuovi pattern: CQRS, Event Sourcing, Materialized View, principio di eventual consistency.

Il design (applicato a Ordering)



Il flusso dei comandi:

"Quando un comando come *PlaceOrder* arriva a Ordering **tramite il gateway API**, il **Command Handler** lo valida. In caso di successo **NON aggiorna direttamente una tabella *orders* tradizionale**. Invece genera uno o più eventi immutabili — *OrderPlacedEvent*, *OrderStatusUpdatedToShippedEvent*. Questi eventi vengono accodati cronologicamente all'**Event Store**, che diventa l'unica fonte di verità per tutti i cambiamenti di stato relativi agli ordini. Si memorizzano solo le azioni che si sono verificate."

Il flusso delle query:

"Query come *GetOrderDetails* o *ListCustomerOrders* arrivano al **Query Handler**, che legge direttamente dal database ottimizzato e dalle viste materializzate. **NON tocca l'event store per le query generali.**"

"Lo stato del read model diventa eventualmente coerente con lo stato dell'event store, per la natura asincrona dell'elaborazione degli eventi."

Scelte tecnologiche

Componente	Opzioni
Event Store (scrittura)	"Deve essere ottimizzato per accodare eventi e riprodurli " → EventStoreDB (dedicato), Apache Kafka come log durevole, Cassandra , o PostgreSQL con tabelle progettate con cura
Read Models (lettura)	Polyglot persistence — "il database migliore per ogni vista": MongoDB per viste flessibili denormalizzate, Elasticsearch per ricerca potente, Redis per caching, Cassandra , DynamoDB , Cosmos DB

I 5 vantaggi

1. **Scalabilità indipendente** ★ — *"Il miglior vantaggio. Si scala l'event store per le scritture in modo diverso dai read model, e si sceglie la migliore tecnologia per ciascuno"*
2. **Audit trail completo** — *"registro immutabile di ogni cambiamento di stato, prezioso per verifica, debug e business intelligence"*
3. **Query temporali** — *"ricostruire lo stato di un'entità come esisteva in qualsiasi momento"*
4. **Read model flessibili** ★ — *"Si possono creare molte viste materializzate diverse dallo STESSO stream di eventi, ognuna adattata a esigenze di query specifiche, senza impattare il modello di scrittura"*
5. **Velocità su entrambi i lati** — *append-only è velocissimo; query su modelli ottimizzati altrettanto*

★ I 5 svantaggi

1. Complessità del sistema

"Più componenti da gestire: event store, event bus, molteplici event consumer, vari database di lettura. Non è un pattern per applicazioni semplici."

2. Eventual consistency dei read model

"Quando il client esegue una query, deve essere preparato a gestire dati potenzialmente non aggiornati."

3. Curva di apprendimento ripida

"Event sourcing, modellazione degli eventi, progettare consumer idempotenti, gestione delle proiezioni: richiedono nuove competenze e diversi modi di pensare."

4. Snapshotting ★

"Per entità con un numero molto elevato di eventi, riprodurre ogni volta tutti gli eventi dall'inizio può diventare lento. Lo snapshotting — salvare periodicamente lo stato corrente — viene spesso usato come ottimizzazione."

Come funziona: se un'entità ha 50.000 eventi, non li rigiochi tutti. Salvi uno snapshot allo stato dell'evento 49.900 e rigiochi solo gli ultimi 100.

5. Evoluzione dello schema degli eventi ★

"Una volta scritti, gli eventi sono IMMUTABILI. Se la struttura dell'evento deve cambiare nel tempo, servono strategie per gestire più versioni di eventi."

Perché è un problema serio: un evento scritto 3 anni fa deve poter ancora essere letto dal codice di oggi. Non puoi "migrare" gli eventi — sono immutabili per definizione. Servono **upcaster**, versioning degli eventi, o tolleranza a schemi multipli.

Problem: Manage Consistency Across Microservices in Distributed Transactions

Dove siamo

*"Abbiamo scalato e ottimizzato i nostri servizi. **Ma ora abbiamo dati distribuiti.** Abbiamo abbracciato Database-per-Service, polyglot persistence, CQRS ed event sourcing."*

★ La domanda

"Cosa succede quando una singola transazione logica di business deve apportare modifiche a diversi microservizi indipendenti, ciascuno con il proprio database?"

"Ad esempio, il successo di un ordine non riguarda solo il servizio Ordering."

Coinvolge anche: Payment (addebito), Inventory (riserva stock), Shipping (spedizione), Notification (conferma). **Cinque database diversi, cinque servizi indipendenti, una sola operazione di business che deve riuscire o fallire insieme.**

Le 3 considerazioni che diventano problemi

Considerazione	Problema
Transazioni distribuite che richiedono di visitare diversi microservizi	Come gestirle senza transazioni ACID globali?
Coerenza tra più microservizi	Come garantirla?
Rollback ed esecuzione di passaggi di compensazione	Cosa succede se uno dei servizi fallisce a metà?

Perché il problema è così duro: nel monolite (Lezione 3) bastava un BEGIN TRANSACTION . . . COMMIT — il database garantiva l'atomicità gratuitamente. Ora hai 5 database diversi, su macchine diverse, forse con tecnologie diverse. **Non esiste un COMMIT che li copra tutti.** E il vecchio protocollo Two-Phase Commit (2PC) è sconsigliato nei microservizi perché blocca risorse e crea accoppiamento temporale.

La soluzione

"Come soluzione, impareremo:"

- **il pattern Saga** per le transazioni distribuite
- **il pattern Transactional Outbox**
- **il pattern delle Compensating Transaction**
- **CDC (Change Data Capture)**

→ **Lezione 12: Distributed Transactions Management**
